

Scalable Graph-Based Data Processing Pipeline Using Docker, Kubernetes, Kafka, and Neo4j

Ashish Nadadur Chakravarthi
anadadur@asu.edu
SCAI, Arizona State University
Tempe, Arizona, USA

Abstract

This project presents the design and deployment of a scalable graph-based data processing pipeline implemented across two phases. Phase 1 focuses on constructing a Dockerized Neo4j environment, loading the NYC Taxi dataset, modeling trip data as a graph, and implementing PageRank and Breadth-First Search (BFS) using the Neo4j Graph Data Science library. Phase 2 extends the system to a cloud-native, real-time streaming architecture using Kubernetes, Apache Kafka, Kafka Connect, and Neo4j, all deployed via Helm. The resulting pipeline continuously ingests filtered taxi trip events, transforms them into graph structures, and enables real-time analytical processing. End-to-end validation demonstrates successful orchestration, streaming integration, and analytical performance. This work highlights the evolution from a single-container graph environment to a distributed streaming graph analytics platform, reflecting modern data engineering practices and scalable system design.

1. Introduction

Modern data-intensive systems increasingly depend on scalable, fault-tolerant, and highly available architectures capable of processing continuous, high-volume data streams. Traditional monolithic deployments, including single-container Neo4j environments, often struggle to meet these requirements due to limited scalability, resource constraints, and a lack of distributed coordination. The objective of this project was to design and implement a fully operational data processing pipeline that evolves from a standalone Neo4j-based environment in Phase 1 to a cloud-native, multi-component streaming architecture in Phase 2.

In Phase 1, the project focused on building a Dockerized Neo4j environment, loading the NYC Taxi dataset, configuring the Graph Data Science (GDS) plugin, and implementing two foundational graph algorithms—PageRank and Breadth-First Search (BFS). This established the basis for performing analytics on real transportation data, modeling locations as nodes and taxi trips as weighted relationships.

Phase 2 expanded on this by implementing a distributed streaming pipeline using Kubernetes. Kafka served as the streaming backbone, Kafka Connect as a transformation layer, and Neo4j as the analytical graph store within the Minikube cluster. Together, these components produced a real-time, horizontally scalable architecture for graph-based analytics. The pipeline enabled continuous ingestion of NYC taxi trip events, their transformation into graph structures, and near real-time execution of GDS algorithms.

2. Methodology

2.1. Phase 1: Neo4j Environment and Graph Construction

Phase 1 involved building a Docker image that installs Neo4j, configures it for external access, loads the NYC Taxi dataset, and installs the GDS plugin. A custom Dockerfile created a working directory (`/cse511/`), installed Java and Python dependencies, downloaded the dataset, and executed a `data_loader.py` script.

The loader created:

- Location nodes labeled `Location`, each representing a pickup or drop-off location ID
- TRIP relationships between locations, with properties: `distance`, `fare`, `pickup_dt`, `dropoff_dt`

These structures formed a weighted directed graph suitable for traversal and ranking.

Two graph algorithms were implemented in `interface.py`:

1. PageRank, using GDS's `gds.pageRank.stream` procedure, was applied to evaluate the relative importance of locations.
2. Breadth-First Search, using `gds.bfs.stream` to find shortest-hop paths between origin and target locations.

Exposed ports (7474 and 7687) enabled direct communication with the Neo4j browser and the Bolt protocol for running algorithmic queries.

Additional Technical Implementation Details

Beyond setting up the basic Neo4j environment, several system-level configurations were required to enable full analytical functionality. The Dockerfile installation included OpenJDK 21 to ensure compatibility with the Graph Data Science (GDS) library, along with Python libraries such as `neo4j`, `pandas`, and `pyarrow` required for dataset ingestion. The `neo4j.conf` file was updated during the image build process to allow Neo4j to listen on all external interfaces (0.0.0.0) and to enable unrestricted APOC and GDS procedures.

The `data_loader.py` script processed the March 2022 NYC Yellow Taxi Parquet dataset using PyArrow and created graph structures using Cypher MERGE operations. Each pickup and dropoff location ID was transformed into a Location node, and each taxi trip produced a TRIP relationship containing distance, fare, pickup_dt, and dropoff_dt properties. These idempotent MERGE operations ensured correct graph construction even when repeated ingest attempts occurred.

Ports 7474 (HTTP) and 7687 (Bolt) were exposed to allow GDS algorithms to be invoked externally via the Neo4j Browser or the Bolt protocol. This configuration enabled the seamless execution of PageRank and BFS during testing.

2.2 Phase 2: Distributed Streaming Pipeline on Kubernetes

Phase 2 introduced a scalable orchestration environment using Minikube and Helm. According to the project specification, the pipeline components were deployed as Kubernetes deployments and services.

Kafka and Zookeeper Setup

Kafka required Zookeeper for coordination. Two YAML manifests were created:

- `zookeeper-setup.yaml` for a Deployment and Service exposing port 2181
- `kafka-setup.yaml` for a Kafka Deployment and Service exposing ports 9092 and 29092

Required configurations included:

```
KAFKA_BROKER_ID=1
```

```
KAFKA_ZOOKEEPER_CONNECT=zookeeper-  
service:2181
```

```
PLAINTEXT_INTERNAL=//kafka-service:29092  
PLAINTEXT://localhost:9092
```

```
Automatic topic creation
```

These settings ensured correct inter-pod communication inside Minikube and external connectivity for producers.

Listener and Networking Configuration

A critical part of the Kafka deployment involved correctly configuring internal and external listeners to support both inter-pod communication and host-level producer access. The configuration defined:

- Internal listener:
`PLAINTEXT_INTERNAL://kafka-
service:29092`

Used by Kafka Connect and other in-cluster services.

- External listener:
`PLAINTEXT://localhost:9092`

Used by the Python producer running outside the cluster.

These listeners were mapped using the `KAFKA_LISTENER_SECURITY_PROTOCOL_MAP` variable and advertised via the `KAFKA_ADVERTISED_LISTENERS` variable. Without this separation, the producer would incorrectly attempt to resolve internal Kubernetes DNS names, resulting in connection failures.

This design ensures stable communication paths inside and outside Minikube, enabling reliable message ingestion and connector consumption.

Neo4j Deployment with Helm

Neo4j was installed using the official Helm chart in standalone mode, with a custom `neo4j-values.yaml` that specifies GDS plugin installation and exposes the Bolt (7687) and HTTP (7474) services through `neo4j-service`. This deployed a fully managed Neo4j pod that accepts streamed writes from Kafka Connect.

Kafka Connect Neo4j Sink Connector

The integration layer was implemented using a Kafka Connect deployment configured through `kafka-neo4j-connector.yaml`. A custom base image (`veedata/kafka-neo4j-connect`) already includes the Neo4j connector JAR files.

A ConfigMap provided the connector JSON configuration, specifying:

- Topic: `nyc_taxicab_data`
- Connector class:
`streams.kafka.connect.sink.Neo4jSinkConnector`

- Authentication credentials
- Cypher transformation query (MERGE-based idempotent writes)

Kafka Connect continuously consumed messages from Kafka and transformed them into Neo4j graph updates.

Connector Authentication Issue and Resolution

During integration, a significant issue was identified in the provided Kafka Connect base image. The container's embedded `sink.neo4j.json` file attempted authentication with Neo4j using the password `project1phase2`, whereas the Neo4j Helm deployment and project specification required the password `processingpipeline`.

This mismatch caused silent authentication failures, preventing the connector from writing nodes and relationships to Neo4j. The issue was resolved by overriding the embedded configuration using a Kubernetes ConfigMap. The ConfigMap supplied a corrected JSON file and was mounted into the container at:

```
/home/appuser/sink.neo4j.json
```

This ensured that Kafka Connect authenticated correctly with Neo4j and enabled the successful ingestion of streaming data.

Data Flow Execution

Data Producer → Kafka (with Zookeeper) → Kafka Connect → Neo4j → Query Execution

Testing involved running the provided `data_producer.py` to stream taxi trip events into Kafka. Port forwarding exposed Kafka and Neo4j outside Minikube for local interaction and validation.

3. Results

3.1 Phase 1 Outcomes

The Neo4j Docker container built successfully and loaded all dataset records. The resulting graph contained:

- 42 Location nodes (matching the expected dataset subset)
- 1,530 TRIP relationships representing individual taxi rides

Tester output confirmed:

- Successful data loading

- 100% passing PageRank test cases
- 100% passing BFS test cases

Both algorithms returned results within milliseconds, validating the correctness and performance of the implementation.

3.2 Phase 2 Deployment Validation

All Kubernetes components reached a `Running` state with stable resource usage. Kafka successfully connected to Zookeeper through internal DNS, and Neo4j accepted Bolt connections from Kafka Connect.

Port forwarding allowed local access to:

- Kafka: 9092
- Neo4j Browser: 7474
- Bolt protocol: 7687

The producer streamed thousands of messages into Kafka; Kafka Connect consumed them, incrementing offsets, validating pipeline throughput.

3.3 Graph and Algorithm Results with Streaming Data

Once the pipeline completed ingestion, Neo4j contained dynamically updated trips and locations for Bronx-filtered NYC taxi data. Running the GDS algorithms on the updated graph yielded:

- BFS results with 1–5 hop paths between the selected source and target nodes
- PageRank scores highlighting the most frequently traversed or centrally connected locations

Computation times remained consistent with Phase 1, hundreds of milliseconds, demonstrating that streaming ingestion did not degrade analytical performance.

4. Discussion

4.1 Architectural Benefits

The transition from single-container Docker to a Kubernetes-managed distributed pipeline produced significant advantages:

- Scalability: Kafka brokers, Neo4j instances, and connectors can be scaled by simply increasing replica counts.

- **Fault Tolerance:** Kubernetes automatically restarts unhealthy pods and ensures service discovery through internal DNS.
- **Loose Coupling:** Kafka decouples producers from consumers, allowing independent scaling and fault isolation.

These characteristics align with modern cloud-native design principles.

4.2 Challenges Faced

A few issues arose during deployment:

- Listener misconfiguration initially caused Kafka to reject producer connections. Correcting `KAFKA_ADVERTISED_LISTENERS` resolved this.
- Kafka Connect readiness timing caused early connector registration failures; adding retry logic fixed the issue.
- Neo4j plugin installation required a correct Helm configuration (`NEO4J_PLUGINS`) to resolve early plugin-loading errors.

These challenges reflect typical complexities of distributed systems, including port management, startup ordering, and environment configuration.

4.3 Future Enhancements

Possible improvements include:

- Horizontal scaling of Kafka brokers and additional Neo4j instances (Enterprise cluster mode).
- Monitoring and observability using Prometheus and Grafana.
- TLS and authentication hardening for inter-service communication.
- Advanced GDS analytics such as community detection, graph embeddings, or similarity algorithms.
- Real-time stream processing using Kafka Streams or Apache Flink before writing to Neo4j.

5. Conclusion

This project demonstrated a comprehensive evolution from a standalone graph-processing environment to a scalable, distributed streaming architecture. Through Docker, Neo4j, Kubernetes, Kafka, Kafka Connect, and GDS, the system

successfully ingested, transformed, and analyzed real-world transportation data in near real time.

Phase 1 established strong foundations in graph modeling, algorithmic computation, and data loading. Phase 2 expanded this into a cloud-native pipeline with automated deployment, orchestration, and continuous streaming capabilities. The resulting architecture is modular, fault-tolerant, and well-suited for advanced applications such as fraud detection, transportation analytics, network optimization, and recommendation systems.

The work showcases how modern distributed data systems can be assembled using containerization and orchestration to achieve high scalability and low latency for graph-based analytics.

References

- [1] Kubernetes Authors, “Kubernetes Documentation” <https://kubernetes.io/docs>
- [2] Apache Software Foundation, “Apache Kafka Documentation” 2025 <https://kafka.apache.org/documentation/>
- [3] Neo4j, Inc., “Neo4j Graph Database Platform,” 2025 <https://neo4j.com/docs/>
- [4] Neo4j, Inc., “Neo4j Graph Data Science Library,” <https://neo4j.com/docs/graph-data-science/>
- [5] Confluent, Inc., “Confluent Platform for Apache Kafka,” <https://docs.confluent.io/platform/current/>
- [6] Helm Authors, “Helm: The Package Manager for Kubernetes,” <https://helm.sh/docs/>